



Xi'an Jiaotong-Liverpool University

西交利物浦大學

**MFE113TC**  
**Programming Foundation for Intelligent  
Manufacturing Engineering**  
**Lecture 04 Flow Control in C/C++**

Dr. Jiachen Liu  
[jiachen.liu@xjtlu.edu.cn](mailto:jiachen.liu@xjtlu.edu.cn)

Fall, 2026

# Outline

- Logic v.s. Common Sense
- Flow Control Branching:
  - Making decisions: with Branching
    - Choosing to do something: **if** ()
    - Choosing between two options: **if** () {...} **else** {...}
    - Choosing between several conditions: **if** () {...} **else if** () {...}
  - Multiple choice menus: with **switch (case / break / default)**;
- Flow Control Looping for Repeating operations: with looping
  - **while** ()
  - **do** {...} **while** ()
  - **for** () {...}

# Logic and Common Sense

## Logic and Statements

- Logic is all about reasoning with statements;
- Statements can be either true or false;
- Logic can be used to deduce new or simpler statements.



# Logic and Common Sense

## Common Sense

- So far, we have taken a procedural view of the logic:

```
if (x is true) do y  
else          do z
```

- Common sense tells us that we can rewrite it as:

```
if (x is false) do z  
else          do y
```

- Because:  $P = \neg(\neg P)$
- But sometimes, **common sense isn't totally reliable** ...;
- IF the logic is wrong, the program will do the wrong thing.

# Logic and Common Sense

## The DeMorgan's Law

- $\neg(P \wedge Q)$ ,  $\neg P \vee \neg Q$ ,  $\neg(P \vee Q)$ ,  $\neg P \wedge \neg Q$

| P | Q | $P \wedge Q$ | $\neg(P \wedge Q)$ | $\neg P$ | $\neg Q$ | $\neg P \vee \neg Q$ | $\neg(P \vee Q)$ | $\neg P \wedge \neg Q$ |
|---|---|--------------|--------------------|----------|----------|----------------------|------------------|------------------------|
| T | T |              |                    |          |          |                      |                  |                        |
| T | F |              |                    |          |          |                      |                  |                        |
| F | T |              |                    |          |          |                      |                  |                        |
| F | F |              |                    |          |          |                      |                  |                        |

# Logic and Common Sense

## The DeMorgan's Law

- $\neg(P \wedge Q)$ ,  $\neg P \vee \neg Q$ ,  $\neg(P \vee Q)$ ,  $\neg P \wedge \neg Q$

| P | Q | $P \wedge Q$ | $\neg(P \wedge Q)$ | $\neg P$ | $\neg Q$ | $\neg P \vee \neg Q$ | $\neg(P \vee Q)$ | $\neg P \wedge \neg Q$ |
|---|---|--------------|--------------------|----------|----------|----------------------|------------------|------------------------|
| T | T | T            | F                  | F        | F        | F                    | F                | F                      |
| T | F | F            | T                  | F        | T        | T                    | F                | F                      |
| F | T | F            | T                  | T        | F        | T                    | F                | F                      |
| F | F | F            | T                  | T        | T        | T                    | T                | T                      |

# Logic and Common Sense

## The DeMorgan's Law

- The negation rule can be proved using a truth table:

| P | Q | P&&Q | !(P&&Q) | !P | !Q | !P  !Q | !(P  Q) | !P&&!Q |
|---|---|------|---------|----|----|--------|---------|--------|
| T | T | T    | F       | F  | F  | F      | F       | F      |
| T | F | F    | T       | F  | T  | T      | F       | F      |
| F | T | F    | T       | T  | F  | T      | F       | F      |
| F | F | F    | T       | T  | T  | T      | T       | T      |

- Similarly, it can be seen that:

$$\mathbf{!(P\&\&Q)\equiv!P||!Q, !(P||Q)\equiv!P\&\&!Q}$$

**—> ! goes into bracket, ||->&&, &&->||**

- These theorems are named after the discoverer **DeMorgan**.

# Logic and Common Sense

## Simplifying C conditions

- When programming, it is usually easier to understand **what conditional statements will do if they are expressed in a positive sense**;
- Examples: with the DeMorgan's Rule to re-write.

```
if (! (w<50 || h<50) )  
if (! (P || Q) )           /*subs P = w<50,Q=h<50*/  
if (!P && !Q)              /*apply DeMorgan's*/  
if (w>=50 && h>=50)       /*subs !P = w>=50,etc*/
```

# Logic and Common Sense

## Simplifying C conditions

- Simplifying the following

```
if (! (p>=200 && p<0) && q>0)
```

- 1) **if** (p<200 && p>=0 && q>0)
  - 2) **if** (p<200 || (p>=0 || q<=0) )
  - 3) **if** ( (p<200 || p<0) || q<=0)
  - 4) **if** (p>=0 || p<200 && q>0)
  - 5) **if** ( (p>=0 || p<200) && q>0)
-

# Logic and Common Sense

## Simplifying C conditions

- Simplifying the following

```
if (! (p >= 200 && p < 0) && q > 0)
```

1) **if** (p < 200 && p >= 0 && q > 0)

2) **if** (p < 200 || (p >= 0 || q <= 0) )

3) **if** ( (p < 200 || p < 0) || q <= 0)

4) **if** (p >= 0 || p < 200 && q > 0) ← && is prior than ||

5) **if** ( (p >= 0 || p < 200) && q > 0) ✓

# Flow Control

## What does the flow control do?

- It enables the program to **follow some particular functions and not others**;
- In general, a programming language must provide 3 types of sequences:
  - **Sequential execution** of statements: ordered processing of statements;
  - Using a **test** to decide between alternative sequences:
    - branching;
    - conditional execution;
  - **Repeating sequences** of statements until a condition is met
    - looping;
    - iterative execution.

# Flow Control

## Flow Control Statements

- Enable you, to make complex decision making, trees, in a program;
- Provide a choice of actions or a repetition of actions;
- Require the evaluation of a **test** expression;
  - A test expression can be either **TRUE** or **FALSE**
  - in C, **0 represents FALSE, non-zero (commonly 1) represents TRUE**;
- Test expressions may involve **relational** and **logical** operators.

# Flow Control

## Flow Control Statements

- Enable you, to make complex decision making, trees, in a program;
- Provide a choice of actions or a repetition of actions;
- Require the evaluation of a **test** expression;
  - A test expression can be either **TRUE** or **FALSE**
  - in C, **0 represents FALSE, non-zero (commonly 1) represents TRUE**;
- Test expressions may involve **relational** and **logical** operators.



<, <=, ==  
!=, >=, >



&&, ||, ...

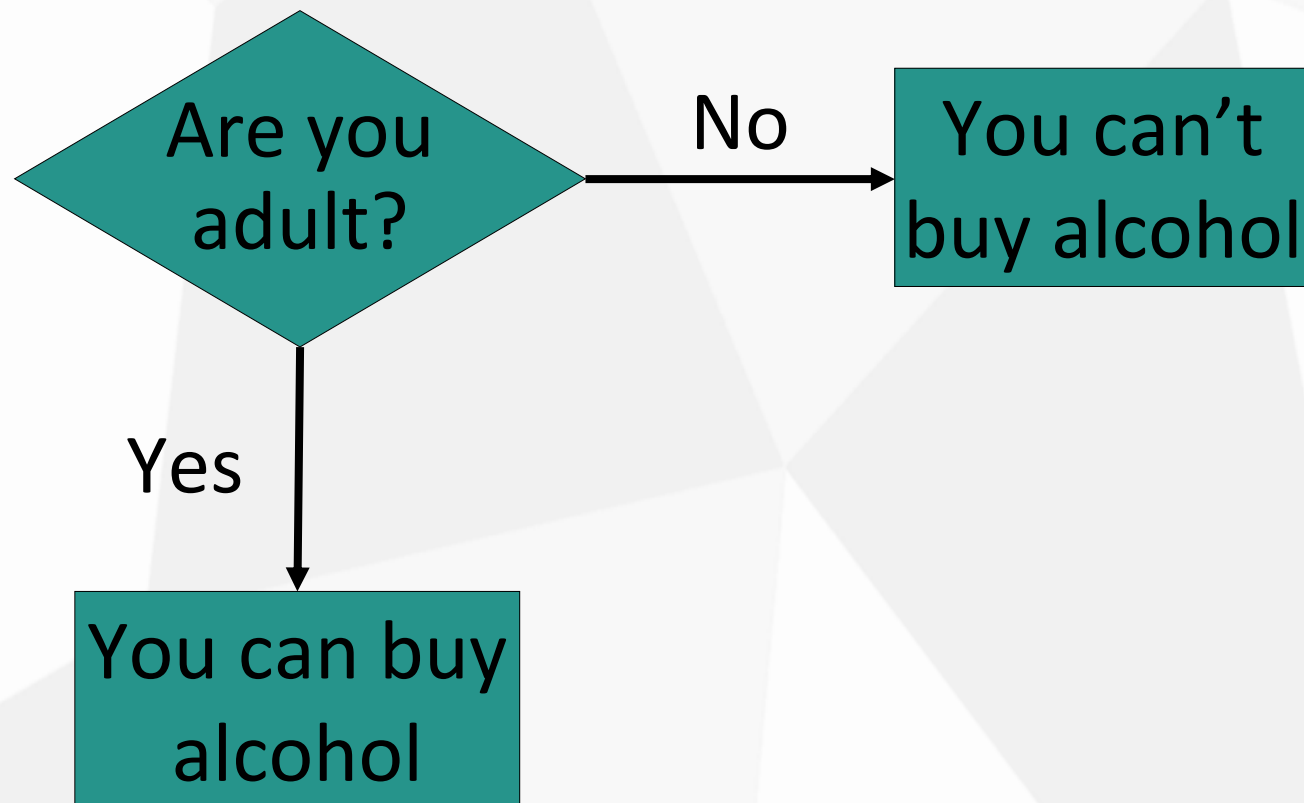
# Flow Control

## Statements and Blocks

- An **expression** (such as `x=0`) is called a statement if it is followed by a semicolon (such as `x=0;`);
  - Each line of the C code needs to be terminated by using a semicolon to make it be a statement;
- Question:
  - Do we still need a semicolon after the curly brackets to group statements?
  - No, we don't.
    - These brackets denote a **functional block or a “compound statement”**;
    - Examples: `main() {...}` —> No semicolon;
    - `if, else, while, for ... {}`, we will see that the **brackets are used to group multiple statements together**.

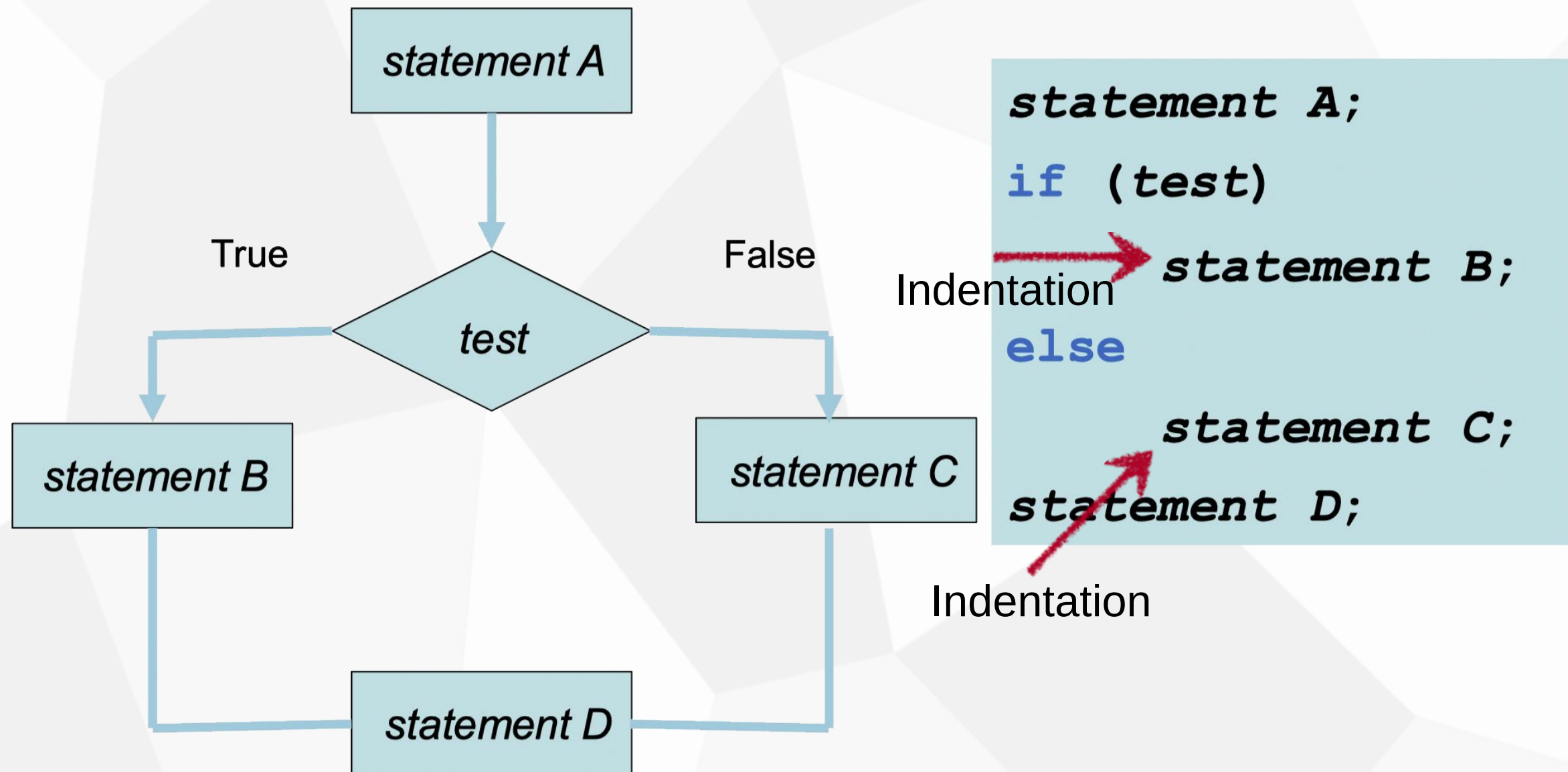
# Choices with the **if** Statement

Remember we can use **if ... else ...** to make choice?



# Choices with the **if** Statement

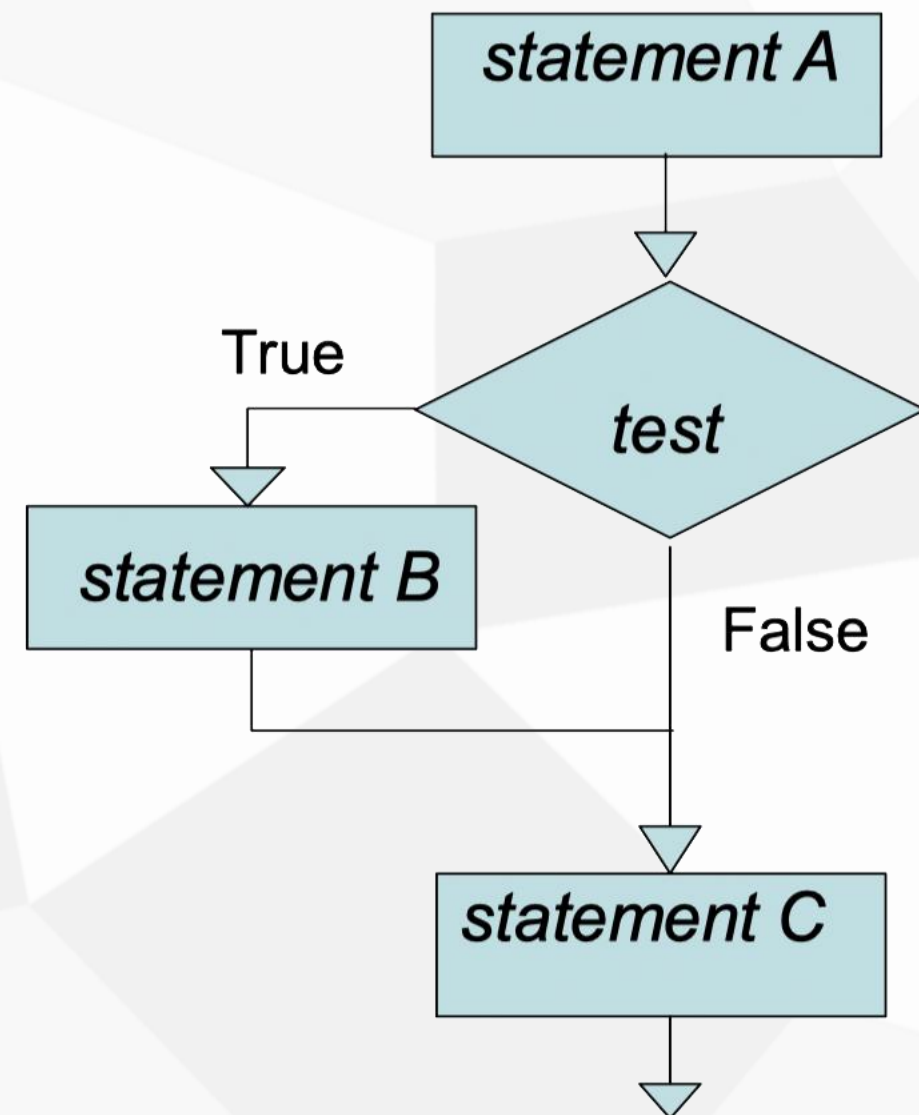
Remember we can use **if ... else ...** to make choice?



# Choices with the **if** Statement

## Choosing to execute a statement

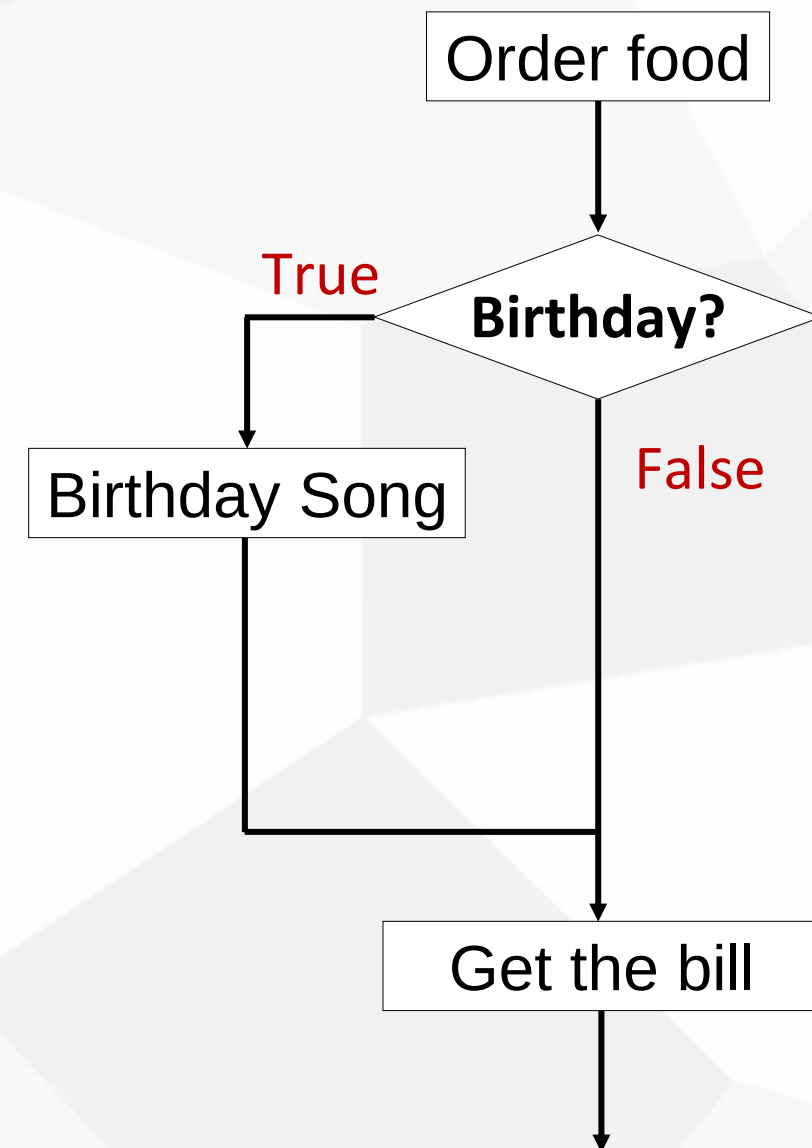
- In our daily life, sometimes we don't need an '**else**'; we just take action when a certain condition is met. If not, nothing happens.



# Choices with the **if** Statement

## Choosing to execute a statement

- In our daily life, sometimes we don't need an '**else**'; we just take action when a certain condition is met. If not, nothing happens.

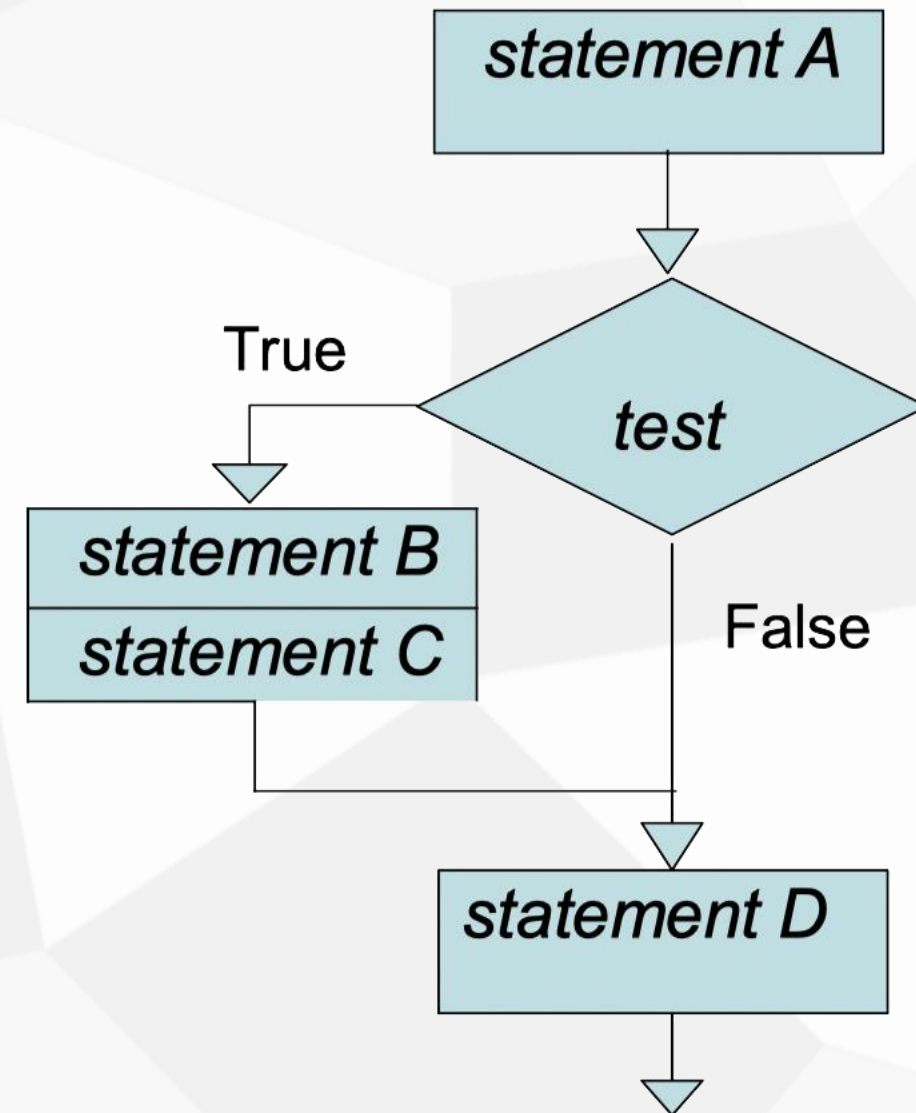


# Choices with the **if** Statement

Choosing to execute a block of statements or not

```
statement A;  
if (test)  
{  
    statement B;  
    statement C;  
}  
statement D;
```

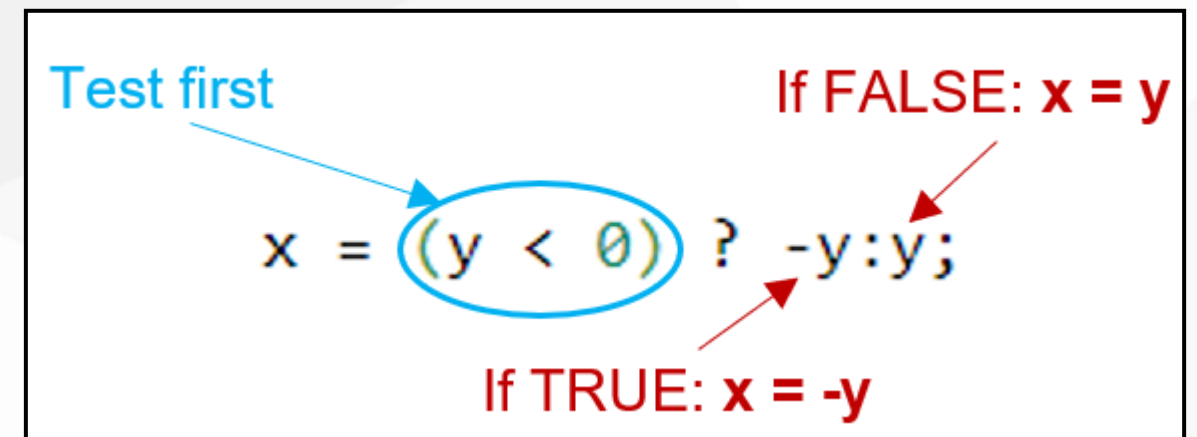
Indentation



# Choices with the **if** Statement

## More **shorthand** for **if else**

- Sometimes, to make the program more compact, engineers write the 'if...else...' statement in a **shorthand** form.
- Remember this approach.



Equivalent

```
if (y < 0)
|     x = -y;
else
|     x = y;
```

# Choices with the **if** Statement

## More shorthand for **if else**

```
#include <stdio.h>

int main()
{
    int x, y;
    printf("Input value for y:\n");
    scanf("%d", &y);

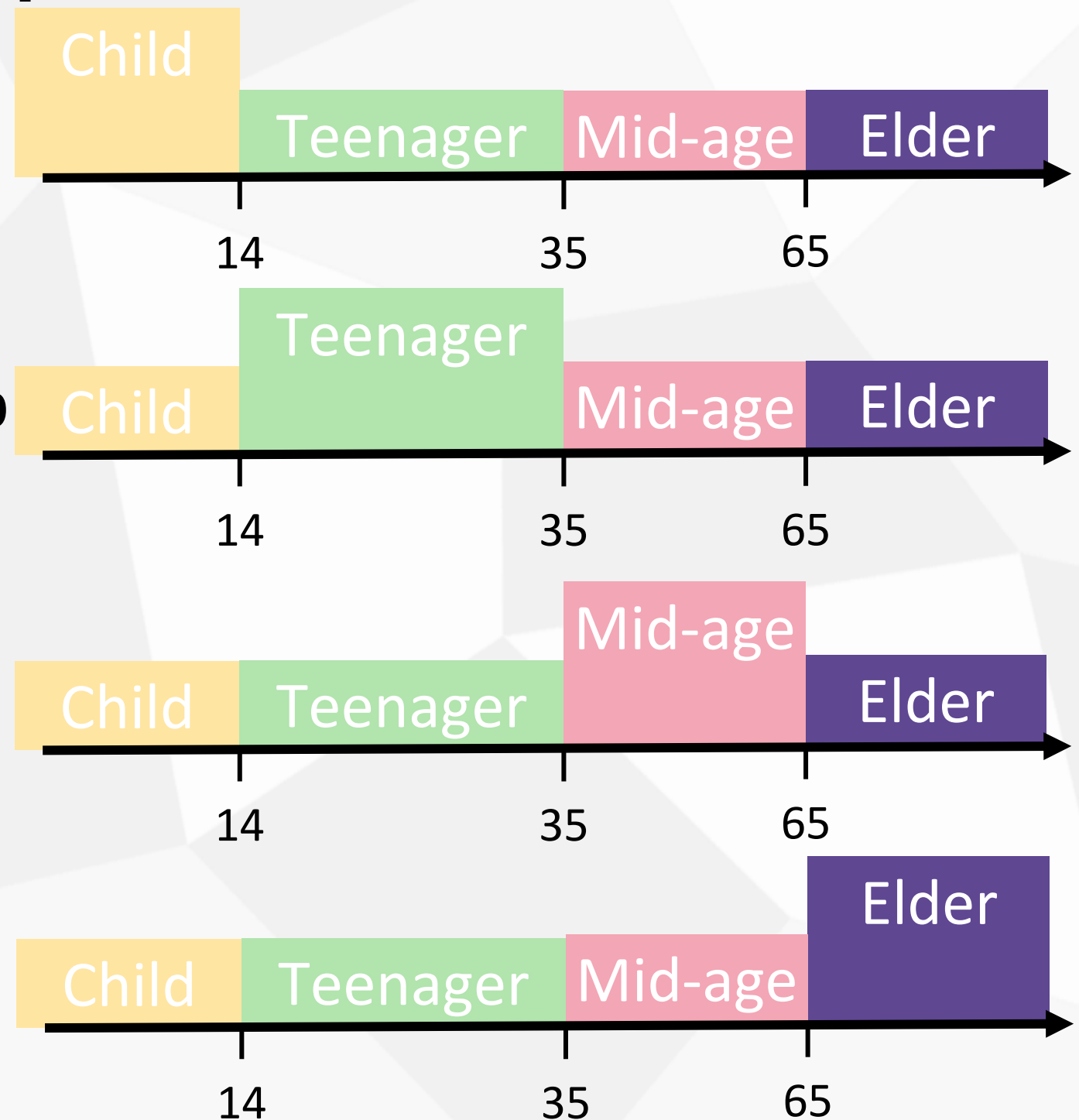
    x = (y<0)? -y:y;

    printf("x's value is %d\n", x);
    return 0;
}
```

# Choices with the **if** Statement

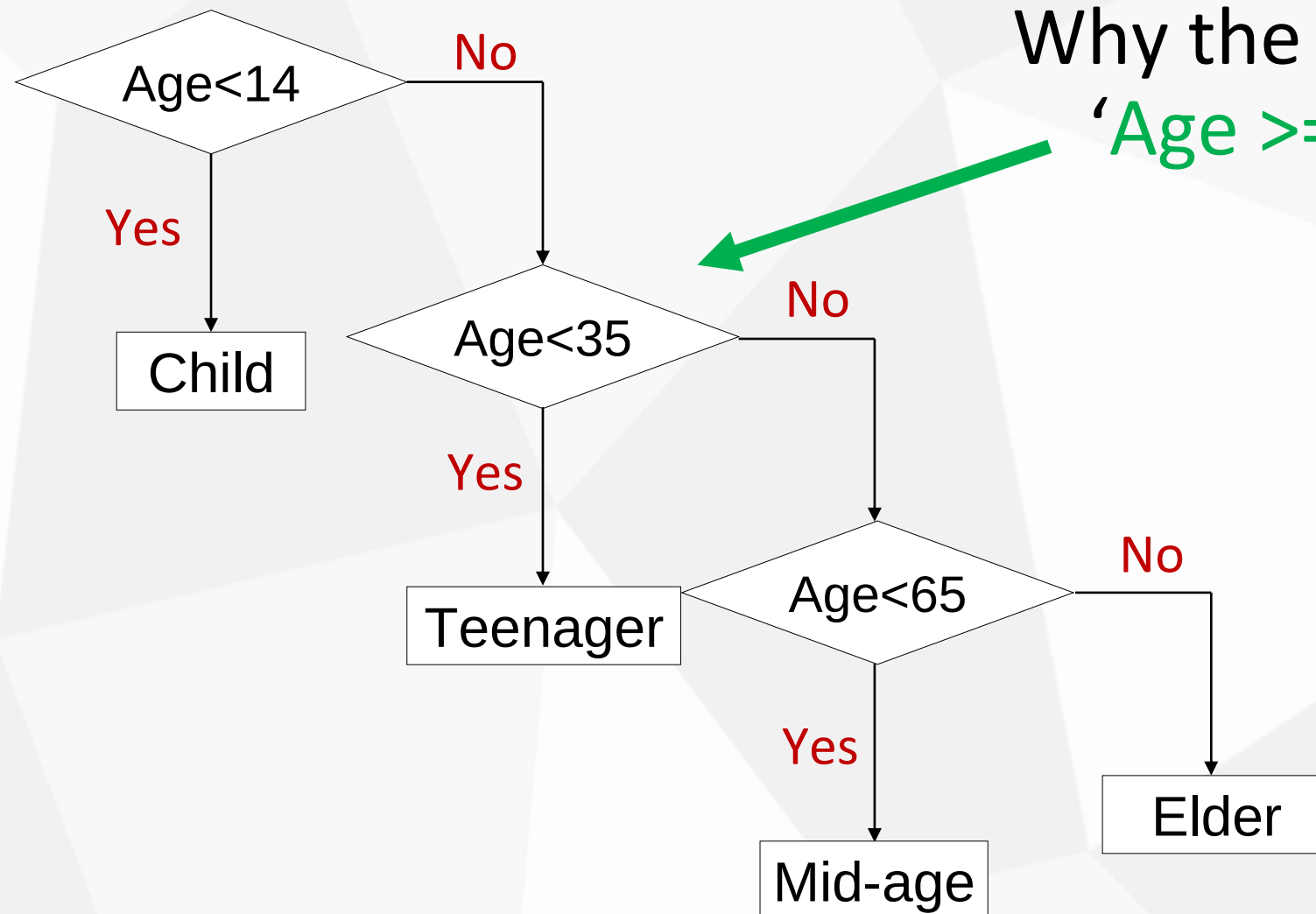
## Choosing between multiple statements or blocks

- Sometimes an event may have multiple options, and we cannot simply use **'if...else'** statements once to handle it.



# Choices with the **if** Statement

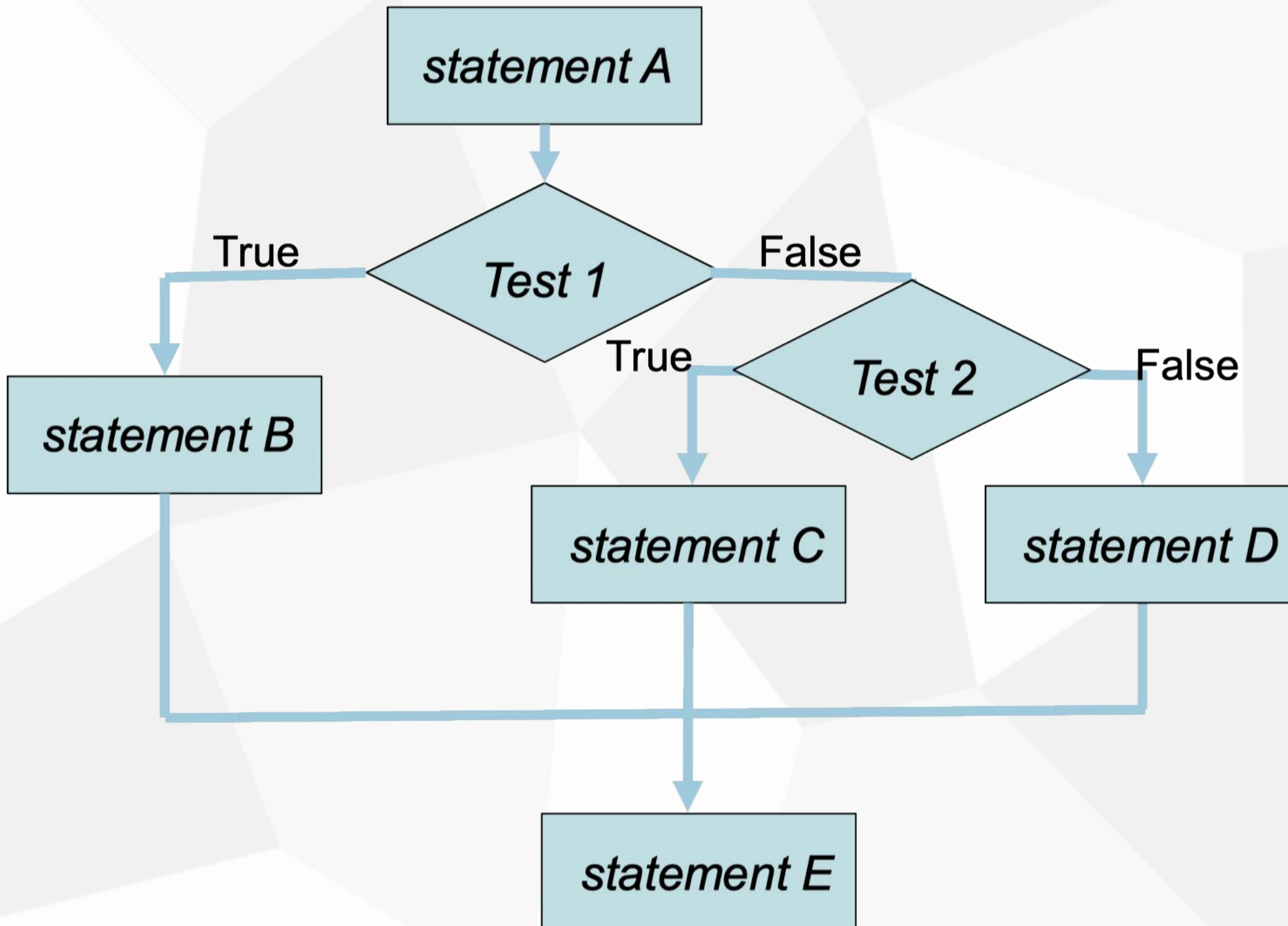
## Choosing between multiple statements or blocks



Why the test statement is not  
'Age >= 14 && Age < 35'?

# Choices with the **if** Statement

## Choosing between multiple statements or blocks



```
statement A;  
if(test1)  
    statement B;  
else if(test2)  
    statement C;  
else  
    statement D;  
  
Statement E;
```

# Choices with the **if** Statement

## Pairs of **if()** and **else**

```
if ( number > 6 )  
    if ( number < 12 )  
        printf("You are close!");  
else  
    printf("You Lose!");
```

- How does this work (which **if** does the **else** belong to)?

| 'number' | output                             |
|----------|------------------------------------|
| 5        | none<br>(does not output anything) |
| 7        | You are close!                     |
| 14       | You Lose!                          |

- else** pairs with the **most recent if** unless braces are used.

# Choices with the **if** Statement

## Pairs of **if()** and **else**

```
if ( number > 6 )  
    if ( number < 12 )  
        printf("You are close!");  
    else  
        printf("You Lose!");
```

Indentation

Indentation

Indentation

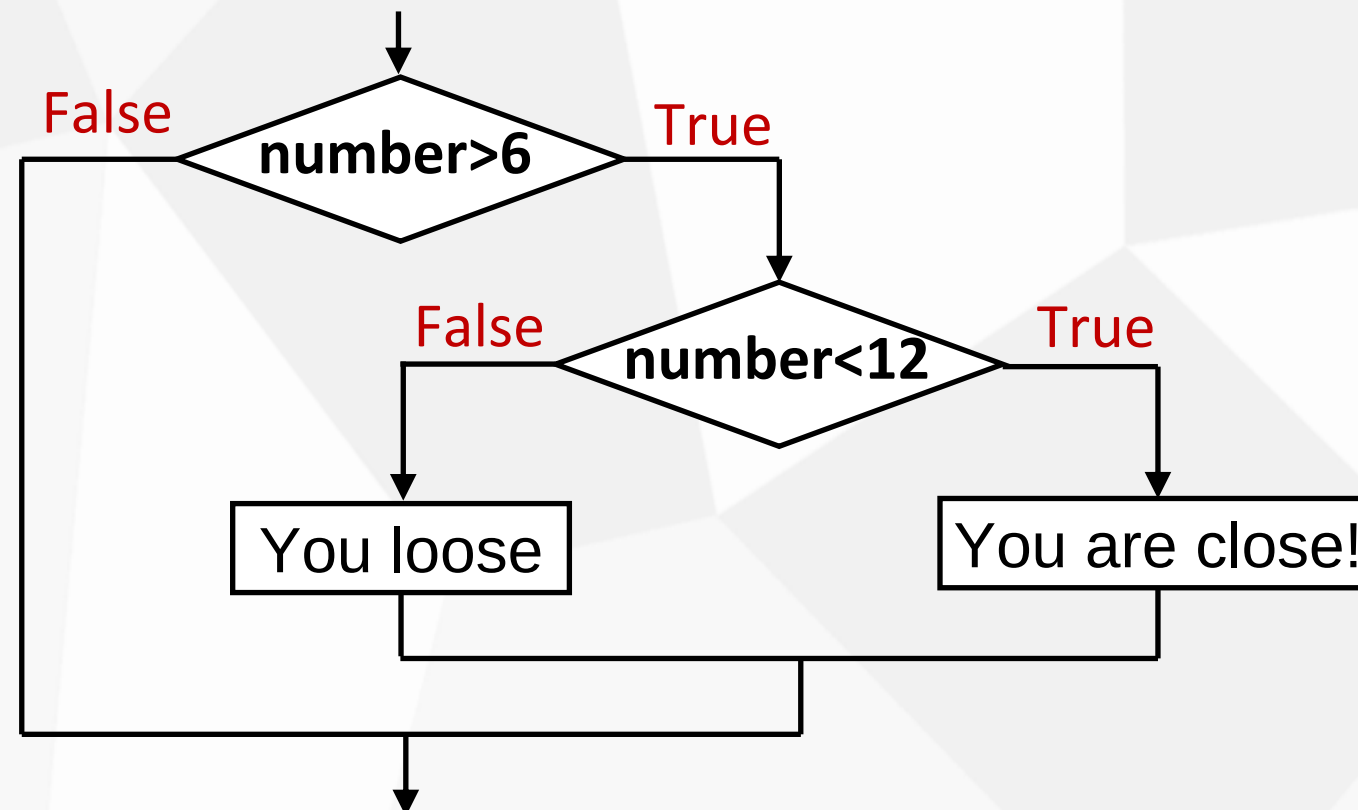
**else** belongs to the closest **if**

- Better to use proper **indentation** to specify the pairs;
- \*Be noted, **we will track your indentation for your projects or assignments !!!**

# Choices with the **if** Statement

## Pairs of **if()** and **else**

```
if (number > 6)
    if (number < 12)
        printf("You are close!");
    else
        printf("You loose");
```

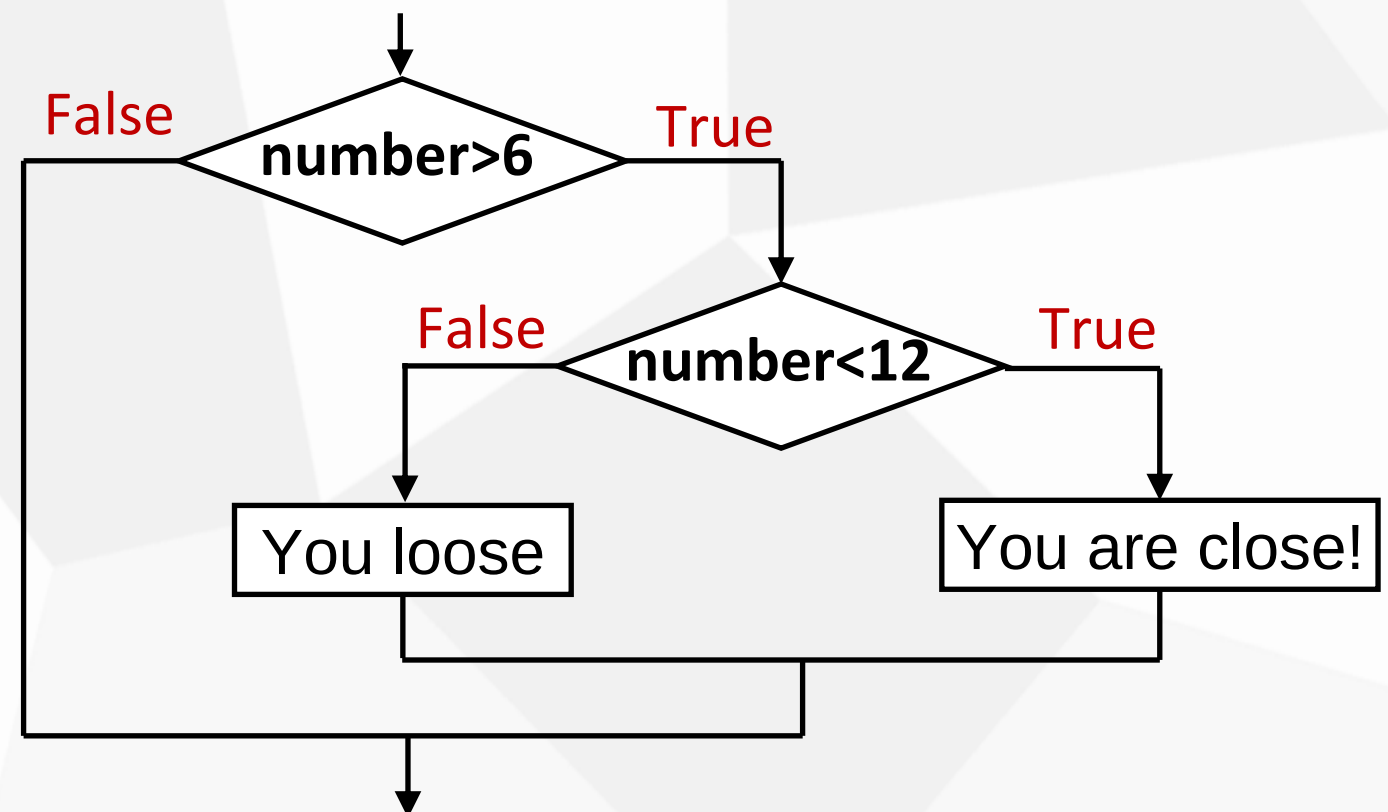


# Choices with the **if** Statement

## Pairs of **if()** and **else**

- It is highly recommended to use curly braces **{ }** to enclose your statements and prevent unintended logic.

```
if (number > 6) {  
    if (number < 12)  
        printf("You are close!");  
else{  
    printf("You lose");  
}
```



# Choices with the **if** Statement

## Pairs of **if()** and **else**

- What is the value of z?

```
int x=5, y=2, z=0;  
if (x > 3)  
    if (y > 2) {  
        z = 1; }  
else if (x < 10) {  
    z = 2; }  
else {  
    z = 3; }
```

- a) 0
- b) 1
- c) 2
- d) 3

# Choices with the **if** Statement

## Pairs of **if()** and **else**

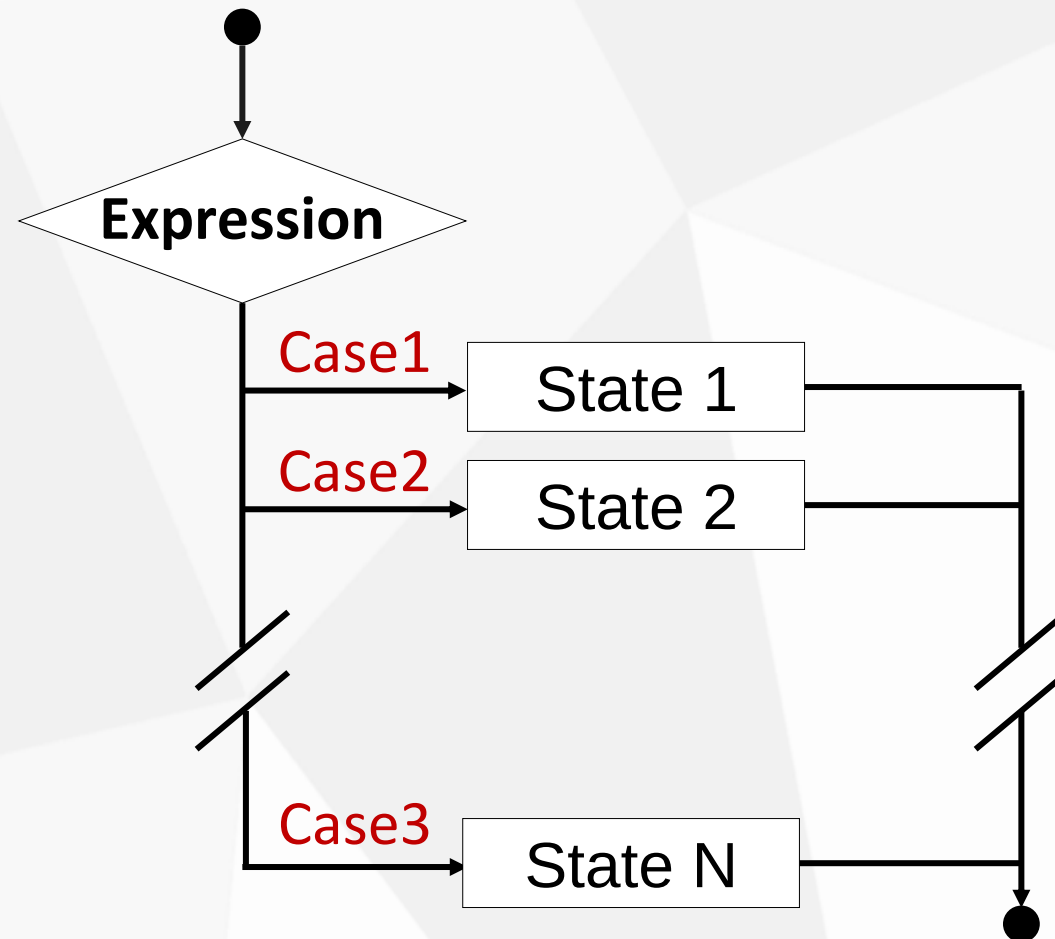
- What is the value of z?

```
int x=5, y=2, z=0;  
if (x > 3)  
    if (y > 2) {  
        z = 1; }  
else if (x < 10) {  
    z = 2; }  
else {  
    z = 3; }
```

- a) 0
- b) 1
- c) 2
- d) 3

# Multiple Choice

with **switch case**, and **break**



- **switch** is used to provide **multiple choices**;
  - Switch test expression must be assigned with integer value;
  - Unlike '**if...else**' which is range-based, case labels must be **integer-type** (constant, variable, or expressions);

# Multiple Choice

## with **switch case**, and **break**

```
switch(integer expression)  
{  
    case constant1: statement1;  
                    break;  
    case constant2: statement2;  
                    break;  
    default : statement3;  
}  
statement4;
```

The integer expression is evaluated, and (say) the result is *constant2*

This means that *statement2* will be executed next

The **break** statement makes the program **exit** the **switch** block and jump to *statement4*

- **switch** is used to provide **multiple choices**;
  - switch test expression must be assigned with integer value;
  - case labels must be **integer-type** (constant, variable, or expressions);
- **break** ends the code block.

# Multiple Choice

## with **switch case**, and **break**

- It is good practice to include **break** in all cases, including the default

```
#include <stdio.h>
#include <simpio.h>

main()
{
    int day;
    scanf("%d", &day);

    switch(day)
    {
        case 0 : printf("Sunday");
                  break;
        case 6 : printf("Saturday");
                  break;
        default: printf("Weekday");
                  break;
    }
}
```

Read integer value - store it in **day**

If **day** is 0 print Sunday

If **day** is 6 print Saturday

The **break** statements make the program exit the **switch** block

For any other number print Weekday

\* Otherwise, without break, the execution of the code will fall through.

# Multiple Choice

## switch without break

```
switch(integer expression)  
{  
  case constant1 : statement1;  
  case constant2 : statement2;  
  case constant3 : statement3;  
  default       : statement4;  
}  
statement5;
```

The integer expression is evaluated and the result is *constant2*

This means that *statement2* will be executed next...

...then *statement3* is executed next...

... then *statement4* is executed next

# Multiple Choice

## Example

```
#include <stdio.h>
int main()
{
    int day;
    scanf ("%d", &day);

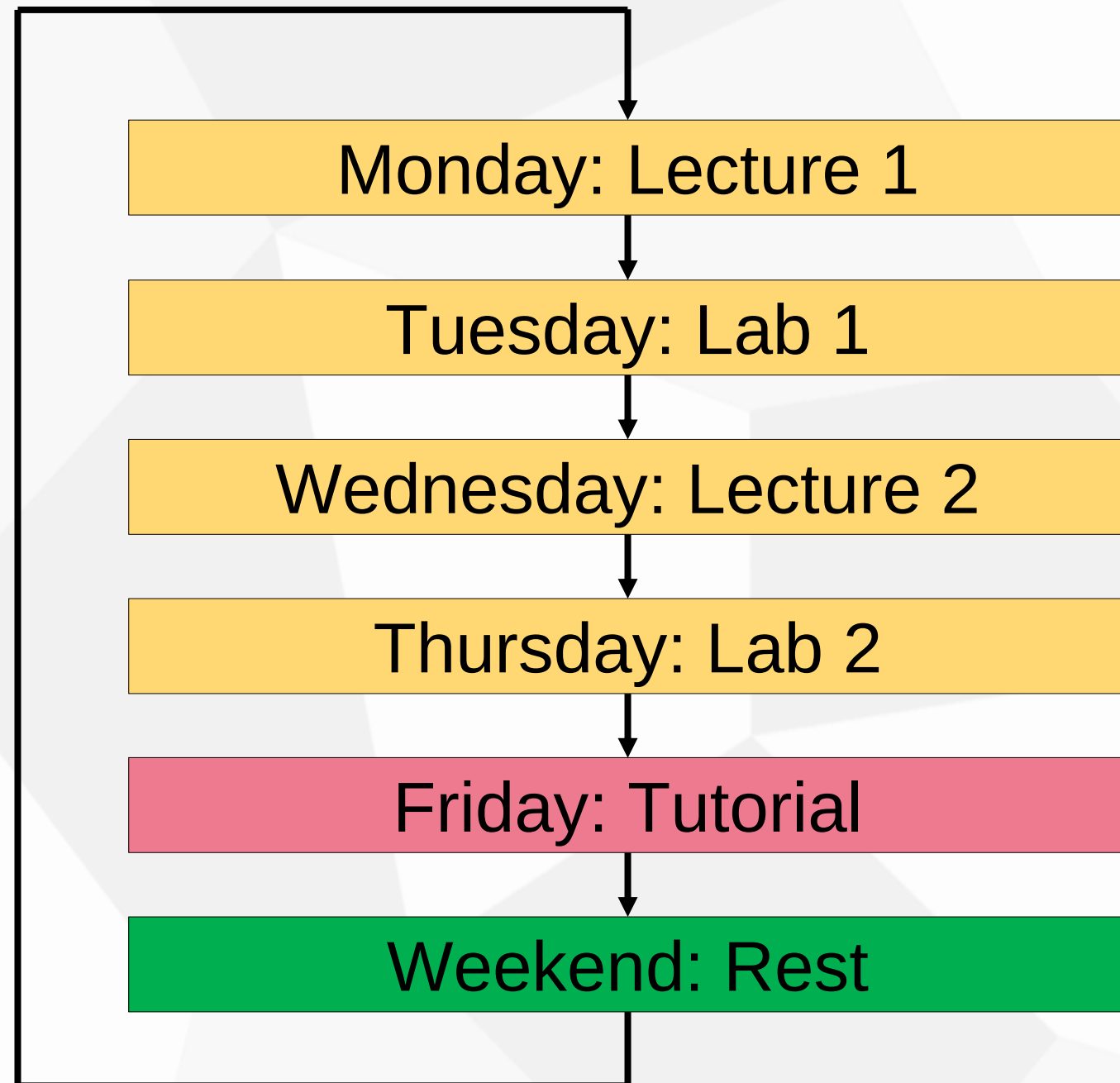
    switch (day) {
        case 0: printf ("Monday\n");
                break;
        case 1: printf ("Tuesday\n");
                break;
        default: printf ("Other day\n");
                break;
    }
    return 0;
}
```

# Questions? Break!



# Looping

What is a loop?



# Looping

## with the **while** loop

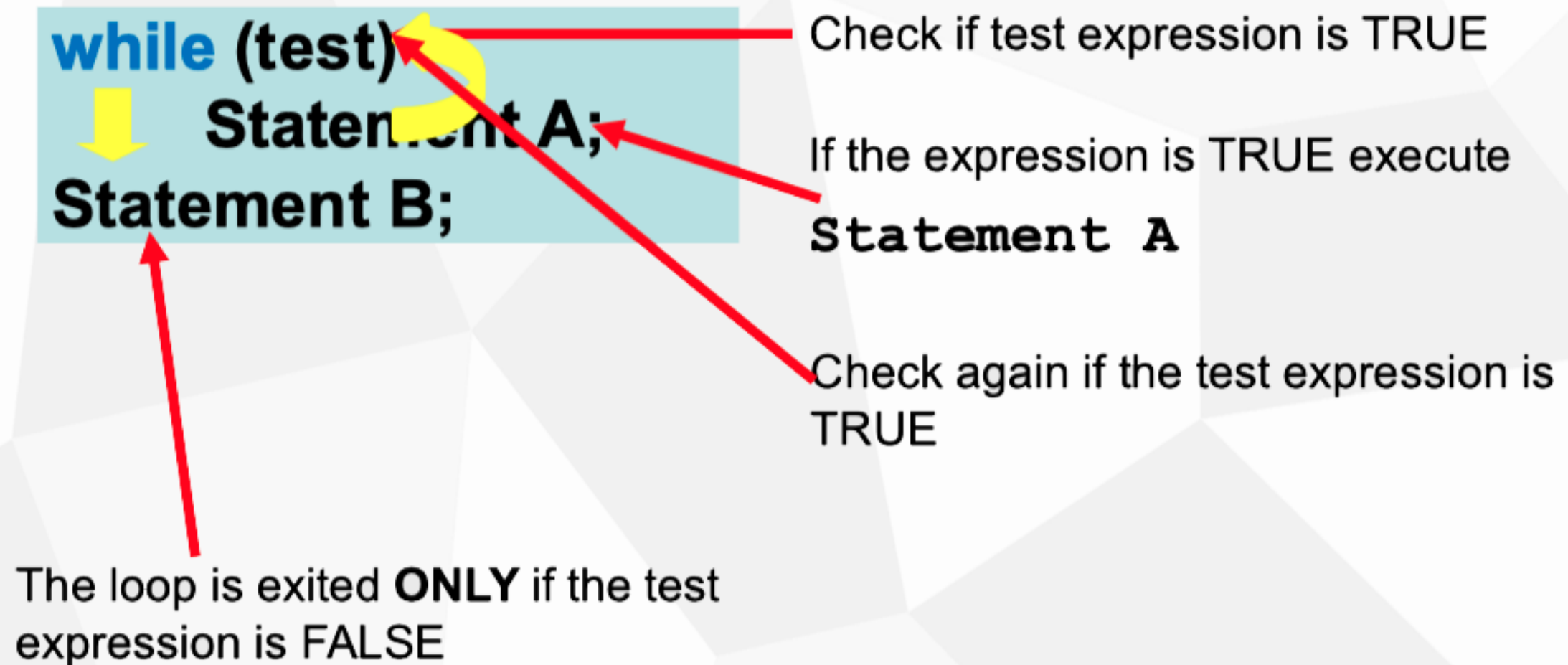
- In C, the syntax for a **while** loop looks like an if statement without **else**;
- The body of the **while** statement is executed repeatedly as long as the condition is true.

```
while (whileCondition) {  
    /* do something */  
}
```

# Looping

## with the **while** loop

- Conditional loop with an **entry condition**:

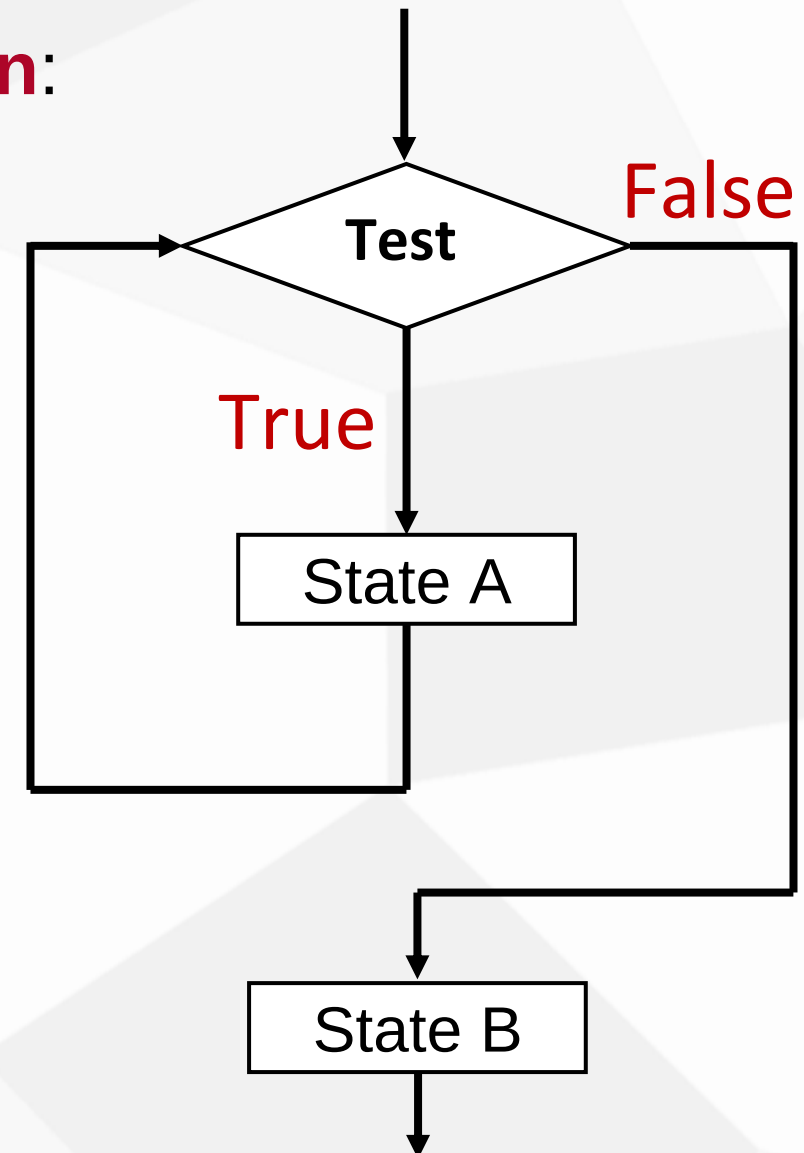


# Looping

## with the **while** loop

- Conditional loop with an **entry condition**:

```
while (test){  
    statement A;  
}  
Statement B;
```



# Looping

## with the **while** loop

```
while(test)
```

```
{  
    statement1;  
    statement2;  
    ...  
}
```

If the test expression is TRUE  
execute the **loop body** (the block of  
statements in between the set of  
squiggly brackets { })

```
int i=0,n,sum=0;  
long factorial=1;  
scanf("%d",&n);
```

```
while(i<n)
```

```
{  
    i = i+1;  
    factorial *= i;  
    sum += i;  
}
```

- Note that part of the loop body  
relates to the loop condition

- It is **good style** to indent the loop  
body statements

Q: What will happen if the loop  
condition is not changed inside  
the loop???

# Looping

## with the **while** loop

```
#include <stdio.h>

int main()
{
    int n=1;
    while (n<5) {
        printf("Still running\n");
    }

    printf("Finished\n");
    return 0;
}
```

# Looping

## The **do-while** loop

- It is an **exit-condition** loop;
- It means that the loop body is executed **at least once**.

```
do  
    statement;  
while (test);
```

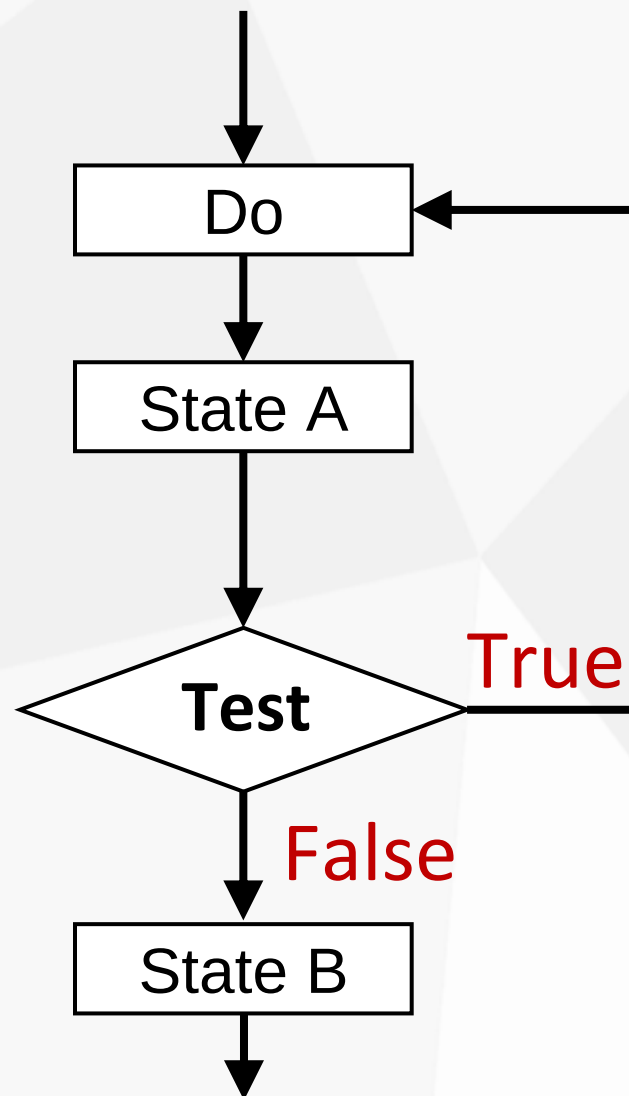
Use the test to decide if you want to have another go ... execute the **statement** or **block of statements** again

```
count = 1;  
do  
{  
    printf("La, la, la!");  
    count ++;  
}  
while (count < 5);
```

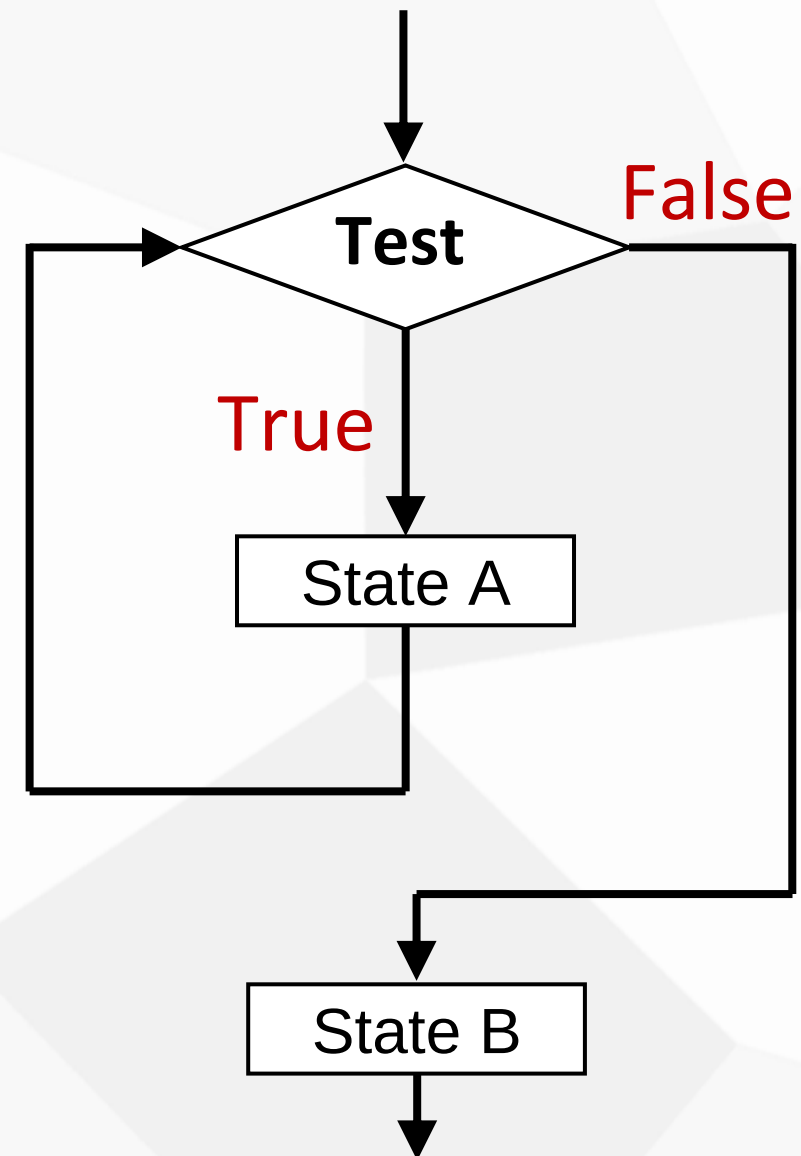
# Looping

The **do-while** or **while** loop?

**do-while**



**while**



# Looping

## The **do-while** or **while** loop?

- Decide if you need an **entry or an exit condition**
  - **Entry condition** loops are preferred;
  - Better to look before leaping;
  - **Entry condition loops make programs easier to read.**
- 
- **So, it means while loop is preferred**

# Looping

## The **do-while** or **while** loop?

- What is the number printed after the loop?

```
float fac = 1.0;  
int n = 0;  
while (1) {  
    n++;  
    if (fac*n > 50)  
        break;  
    fac = fac * n;  
}  
printf("Factorial = %f ", fac);
```

- a) 1
- b) 2
- c) 6
- d) 24
- e) 120

- **break**: end the entire loop immediately !
- **continue**: stop the current loop and go to the next loop.

# Looping

## The **do-while** or **while** loop?

- What is the number printed after the loop?

```
float fac = 1.0;  
int n = 0;  
while (1) {  
    n++;  
    if (fac*n > 50)  
        break;  
    fac = fac * n;  
}  
printf("Factorial = %f ", fac);
```

- a) 1
- b) 2
- c) 6
- d) 24
- e) 120

- break**: end the entire loop immediately !
- continue**: stop the current loop and go to the next loop.

# Looping

## The **for** Loop

```
for ( init; test; step )  
    statement;  
next statement;
```

```
for ( i=1; i<=10; i++ )  
    factorial*=i;  
printf ("%d", factorial);
```

- Combines 3 actions in one place:
  - **init**: initialise the **counter variable**;
  - **test**: logical condition;
  - **step**: modify the counter variable.

# Looping

## The **for** Loop

```
for (init; test; step)  
    statement;  
next statement;
```



```
init;  
while (test)  
{  
    statement;  
    step;  
}  
next statement;
```

Equivalent to



```
for ( i=1; i≤10; i++ )  
    factorial*=i;  
printf("%d", factorial);
```

```
i=1;  
while ( i≤10 )  
{  
    factorial*=i;  
    i++;  
}  
printf("%d", factorial);
```

# Looping

## The **for** Loop

- What is the number printed after the loop?

```
int n, total=0;  
for (n=1; n<5; n++)  
    total = total + n*n;  
  
printf("Total: %d\n", total);
```

- a) 5
- b) 10
- c) 16
- d) 25
- e) 30

# Looping

## The **for** Loop

- What is the number printed after the loop?

```
int n, total=0;  
for (n=1; n<5; n++)  
    total = total + n*n;  
  
printf("Total: %d\n", total);
```

- a) 5
- b) 10
- c) 16
- d) 25
- e) 30

# Looping

## Comparison: Indefinite (**while**) v.s. Counting (**for**)

- **Indefinite loops:** do not know in advance how many times the loop will be executed;
- **Counting loops:** the loop is executed for a fixed (known) number of times.
  - A counter should be initialised;
  - The counter is compared with a limiting value;
  - The counter is incremented each time the loop is completed.

# Looping

## The **for** Loop: Customised update expression

```
#include <stdio.h>
```

```
main()
```

```
{
```

```
    double debt;
```

```
    for (debt=100.0; debt < 150; debt*=1.1)
```

```
        printf ("Your debt is now %.2f\n", debt);
```

```
}
```

This time the **update expression** involves some extra arithmetic calculations.



*Program output*

```
Your debt is now 100.00  
Your debt is now 110.00  
Your debt is now 121.00  
Your debt is now 133.10  
Your debt is now 146.41
```

# Looping

## The **for** Loop: Counting Down

In this example our counting variable is **decremented** every time the loop is executed

```
#include <stdio.h>
main()
{
    int secs;
    for(secs=5;secs>0;secs--)
        printf("%d seconds\n",secs);
    printf("We have ignition!");
    return 0;
}
```

### *Program output*

```
5 seconds
4 seconds
3 seconds
2 seconds
1 seconds
We have ignition!
```

# Looping

## The **for** Loop: increment options

In this example our counter variable is **increased by 12** every time the loop is executed

```
#include <stdio.h>
main()
{
    int n;
    for (n=0 ; n<55 ; n=n+12)
        printf("%d \n", n) ;
}
```

*Program output*

0  
12  
24  
48

# Looping

## The **for** Loop: by character

- Remember that characters are just numbers

### ASCII code

. . .

```
#include <stdio.h>
main()
{
    char ch;
    for (ch='a' ; ch<='z' ; ch++)
        printf("ASCII value of %c is %d\n", ch, ch);
}
```

Here we compare and increment the numeric code for the character stored in **ch** variable

# Looping

## The **for** Loop: alternating conditions

```
#include <stdio.h>
```

```
main()
```

```
{
```

```
    int n;
```

```
    for (n=0;n*n < 12; n++)
```

```
        printf ("%d \n", n);
```

```
}
```

Our **test expression** involves some extra arithmetic calculations. Do you remember which operator is **evaluated first** – has the **higher**

**precedence?**

*Program output*

0

1

2

3

# Looping

## The **for** Loop: alternating conditions

- Any of the three expressions in the **for** statement can be left out;
  - if expression 1 (init) or 3 (step) are omitted, then they just don't happen;
  - if expression 2 (test) is omitted then the loop appears **permanently true**.

Here we have omitted the expression that **tests** the condition, **the loop will never be terminated!**

```
#include <stdio.h>
main()
{
    int time = 5;
    for (n=2;; time = time*n)
        printf("n=%d; time is %d.", n, time);
}
```

Q: Then what will happen???

# Looping

## The **for** Loop: nested loops

- A nested loop is a loop inside another loop;
- It is used to display data as rows and columns.

```
#include <stdio.h>
#define ROWS 4
#define CHARS 4

main ()
{
    int row; char ch;
    for (row = 0; row < ROWS; row++)
    {
        for (ch = ('A' + row); ch < ('A' + CHARS); ch++)
            printf ("%c", ch);
        printf ("\n");
    }
}
```

*Program output*

```
ABCD
BCD
CD
D
```

# Looping

## The **for** Loop: nested loops

- Sometimes it is useful to use an indicate loop and **make the termination test inside the loop** using **break** or **continue**;
- Example:

```
int p = 1000000;  
while (1) {  
    p++;  
    if (P>20)  
        break;  
}  
  
/* could also use: for (; ;) */  
/* 1st attempt is 1000001 */  
  
/* break when P is greater than 20 */
```

# Looping

## The **for** Loop: Mid-Test Loop

- **break**: exit the loop immediately;
- **continue**: skip the rest of current looping cycle and start a new looping cycle.

```
char ch;  
for(int i=0;i<10;i++)  
{  
    ch = getchar();  
    if (ch == '\n') {  
        continue;  
    }  
    putchar(ch);  
}
```

```
char ch;  
for(int i=0;i<10;i++)  
{  
    ch = getchar();  
    if (ch == 'q') {  
        break;  
    }  
    putchar(ch);  
}
```

# Looping

## The **for** Loop

- The loop should print all the numbers between 1 and 100. What is the wrong with it?

```
int k;  
for (k=1; k!=100; k=k+2)  
{  
    printf ("Odd: %d \n", k);  
    printf ("Even: %d \n", (k+1));  
}
```

- a) k=1;
- b) k!=100;
- c) K=k+2;
- d) "Odd:%d\n"
- e) (k+1)

# Looping

## The **for** Loop

- The loop should print all the numbers between 1 and 100. What is the wrong with it?

```
int k;  
for (k=1; k!=100; k=k+2)  
{  
    printf ("Odd: %d \n", k);  
    printf ("Even: %d \n", (k+1));  
}
```

99, 101; never reach 100

- a) k=1;
- b) k!=100;**
- c) K=k+2;
- d) "Odd:%d\n"
- e) (k+1)

# Thanks